

Execution Guidelines for the Proposed System

1. System Requirements

Before executing the proposed cheiloscopy recognition system, the following software and hardware requirements must be satisfied.

Hardware Requirements

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: At least 5 GB free space
- GPU: Optional but recommended for faster training

Software Requirements

- Operating System: Windows / Linux
- Python Version: Python 3.8 or above
- Development Environment: Visual Studio Code or any Python IDE
- Libraries:
 - TensorFlow
 - OpenCV
 - MediaPipe
 - NumPy
 - Streamlit
 - tqdm
 - keras-cv-attention-models

These libraries can be installed using the following command:

```
pip install tensorflow opencv-python mediapipe numpy streamlit tqdm keras-cv-attention-models
```

2. Project Folder Structure

The project must be organized in the following directory structure for proper execution.

```
cheiloscopy/
├── dataset/
├── preprocessing/
│   ├── extract_lip_region.py
│   ├── preprocess_lip_print.py
│   └── run_preprocessing.py
├── pair_generation/
│   ├── augment.py
│   ├── generate_pairs.py
│   └── run_pair_generation.py
├── processed_data/
├── pairs/
├── model/
│   ├── siamese_model.py
│   ├── train.py
│   ├── evaluate.py
│   └── rank_evaluate.py
├── demo.py
└── requirements.txt
```

3. Dataset Preparation

The dataset must contain **individual folders for each person**, where each folder includes:

- One **face image**
- One **lip print image**

Example dataset structure:

```
dataset/  
├── person_01/  
│   ├── face.png  
│   └── lip.png  
└── person_02/  
    ├── face.png  
    └── lip.png
```

The **face image** is used to extract the lip region, and the **lip image** represents the lip print sample.

4. Execution Pipeline

The proposed system is executed in **four main stages**:

1. **Preprocessing**
2. **Pair Generation**
3. **Model Training**
4. **Model Evaluation and Identification**

Step 1: Data Preprocessing

This step extracts the **lip region from the face image** and enhances the **lip print image**.

Execute the preprocessing script:

```
python preprocessing/run_preprocessing.py
```

The output will be stored in the `processed_data` folder as `.npy` files.

Step 2: Pair Generation

In this step, the system generates **positive and negative training pairs** using augmented images.

Execute:

```
python pair_generation/run_pair_generation.py
```

The following files will be generated:

```
pairs/  
X1.npy  
X2.npy  
y.npy
```

Where:

- `X1.npy` → Lip print images
- `X2.npy` → Lip region images
- `y.npy` → Pair labels (1 = same person, 0 = different person)

Step 3: Model Training

The Siamese neural network is trained using the generated image pairs.

Execute:

```
python model/train.py
```

After training, the model will be saved as:

```
model/cheiloscopy_siamese_model.h5
```

Step 4: Model Evaluation

The trained model is evaluated to measure identification performance.

Ranking-based evaluation can be performed using:

```
python model/rank_evaluate.py
```

Step 5: Running the User Interface

A graphical interface is implemented using **Streamlit** to simulate crime investigation.

Run the interface using:

```
streamlit run demo.py
```

The interface allows the user to:

1. Upload a **crime scene lip print**
2. Upload **suspect images**
3. Generate **ranked similarity results**

5. Output of the System

The system outputs:

- Similarity distance between the crime lip print and suspect images
- Ranked list of suspects based on similarity
- Identification of the most probable match

CODE

1.preprocessing

```
import cv2
import numpy as np
import mediapipe as mp

mp_face_mesh = mp.solutions.face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True
)

LIP_LANDMARKS = [
    61, 146, 91, 181, 84, 17,
    314, 405, 321, 375, 291, 308
```

```
]
```

```
def extract_lip_region(image_path):

    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Unable to read image: {image_path}")

    h, w, _ = img.shape
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = mp_face_mesh.process(rgb)

    if not results.multi_face_landmarks:
        raise ValueError("No face detected")

    xs, ys = [], []
    for idx in LIP_LANDMARKS:
        lm = results.multi_face_landmarks[0].landmark[idx]
        xs.append(int(lm.x * w))
        ys.append(int(lm.y * h))

    x1, x2 = max(0, min(xs)), min(w, max(xs))
    y1, y2 = max(0, min(ys)), min(h, max(ys))

    lip_crop = img[y1:y2, x1:x2]
    lip_crop = cv2.resize(lip_crop, (224, 224))
    lip_crop = lip_crop.astype(np.float32) / 255.0

    return lip_crop
```

```
import cv2
import numpy as np
```

```
def preprocess_lip_print(image_path):
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Unable to read image: {image_path}")

    img = cv2.resize(img, (224, 224))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)

    blurred = cv2.GaussianBlur(enhanced, (5, 5), 0)
    normalized = blurred.astype(np.float32) / 255.0

    return normalized
```

```
import os
import numpy as np
from tqdm import tqdm
```

```

from preprocess_lip_print import preprocess_lip_print
from extract_lip_region import extract_lip_region

DATASET_DIR = r"C:\Users\ADMIN\Desktop\cheiloscopy\dataset"
OUTPUT_DIR = r"C:\Users\ADMIN\Desktop\cheiloscopy\processed_data"

os.makedirs(OUTPUT_DIR, exist_ok=True)

persons = sorted(os.listdir(DATASET_DIR))

for person in tqdm(persons, desc="Processing dataset"):
    person_path = os.path.join(DATASET_DIR, person)

    face_path = os.path.join(person_path, "face.png")
    lip_print_path = os.path.join(person_path, "lip.png")

    if not os.path.exists(face_path) or not os.path.exists(lip_print_path):
        print(f"Skipping {person}: Missing files")
        continue

    try:
        lip_print = preprocess_lip_print(lip_print_path)
        lip_region = extract_lip_region(face_path)

        out_person_dir = os.path.join(OUTPUT_DIR, person)
        os.makedirs(out_person_dir, exist_ok=True)

        np.save(os.path.join(out_person_dir, "lip_print.npy"), lip_print)
        np.save(os.path.join(out_person_dir, "lip_region.npy"), lip_region)

    except Exception as e:
        print(f"Error processing {person}: {e}")

print("Preprocessing completed successfully.")

```

2.pair generation

```

import numpy as np
import cv2
import random

def augment_image(img):
    # Resize to 224x224 if needed
    img = cv2.resize(img, (224, 224))

    # Ensure grayscale
    if img.ndim == 3 and img.shape[-1] == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Horizontal flip
    if random.random() < 0.5:
        img = cv2.flip(img, 1)

```

```

# Small rotation
angle = random.uniform(-10, 10)
h, w = img.shape
M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1)
img = cv2.warpAffine(img, M, (w, h))

# Blur
img = cv2.GaussianBlur(img, (3,3), 0)

# Normalize to [0,1]
img = img.astype(np.float32) / 255.0

# Convert grayscale to RGB for MobileViT
img = cv2.cvtColor((img*255).astype(np.uint8), cv2.COLOR_GRAY2RGB) # shape
(224,224,3)
img = img.astype(np.float32) / 255.0

return img

import os
import numpy as np
import random
from augment import augment_image

def load_data(processed_dir):
    data = {}
    for person in os.listdir(processed_dir):
        path = os.path.join(processed_dir, person)
        lp = np.load(os.path.join(path, "lip_print.npy"))
        lr = np.load(os.path.join(path, "lip_region.npy"))
        data[person] = (lp, lr)
    return data

def generate_pairs(data, augmentations=5):
    X1, X2, y = [], [], []
    persons = list(data.keys())

    for person in persons:
        lp, lr = data[person]

        # Positive pairs
        for _ in range(augmentations):
            X1.append(augment_image(lp))
            X2.append(augment_image(lr))
            y.append(1)

        # Negative pairs
        for _ in range(augmentations):
            neg_person = random.choice([p for p in persons if p != person])
            _, neg_lr = data[neg_person]
            X1.append(augment_image(lp))
            X2.append(augment_image(neg_lr))
            y.append(0)

```

```

        return np.array(X1), np.array(X2), np.array(y)

import numpy as np
from generate_pairs import load_data, generate_pairs

PROCESSED_DIR = r"C:\Users\ADMIN\Desktop\cheiloscopy\processed_data"
OUTPUT_DIR = r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs"

import os
os.makedirs(OUTPUT_DIR, exist_ok=True)

data = load_data(PROCESSED_DIR)
X1, X2, y = generate_pairs(data, augmentations=8)

np.save(f"{OUTPUT_DIR}/X1.npy", X1)
np.save(f"{OUTPUT_DIR}/X2.npy", X2)
np.save(f"{OUTPUT_DIR}/y.npy", y)

print("Step 3 completed")
print("Pairs created:", len(y))

```

3. Model training

```

# siamese_model.py
import tensorflow as tf
from tensorflow.keras import layers, models
from keras_cv_attention_models import mobilevit

def build_encoder(input_shape=(224, 224, 3)):
    """
    Encoder using MobileViT backbone.
    Input: RGB images (224x224x3)
    Output: 128-dimensional embedding
    """
    inputs = layers.Input(shape=input_shape)

    # MobileViT backbone (small) for feature extraction
    base_model = mobilevit.MobileViT_S(
        input_shape=input_shape,
        pretrained=True,
        num_classes=0 # disables top classification layer
    )
    x = base_model(inputs)

    # Global average pooling + embedding projection
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.BatchNormalization()(x)

    return models.Model(inputs, x, name="Encoder")

def build_siamese_network(input_shape=(224, 224, 3)):

```

```

"""
Build a Siamese network using the MobileViT encoder.
"""

encoder = build_encoder(input_shape)

# Two inputs for the Siamese network
input_1 = layers.Input(shape=input_shape)
input_2 = layers.Input(shape=input_shape)

# Pass through shared encoder
emb_1 = encoder(input_1)
emb_2 = encoder(input_2)

# L1 distance
distance = layers.Lambda(lambda tensors: tf.abs(tensors[0] - tensors[1]))([emb_1,
emb_2])
output = layers.Dense(1, activation='sigmoid')(distance)

model = models.Model([input_1, input_2], output)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

return model

# train_siamese.py
import numpy as np
import os
from siamese_model import build_siamese_network

# -----
# Load pairs of images
# -----
X1 = np.load(r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs\X1.npy")
X2 = np.load(r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs\X2.npy")
y = np.load(r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs\y.npy")

# -----
# Convert grayscale (1-channel) to RGB (3-channel) for MobileViT
# -----
if X1.shape[-1] == 1:
    X1 = np.repeat(X1, 3, axis=-1)
    X2 = np.repeat(X2, 3, axis=-1)

print("X1 shape (RGB):", X1.shape)
print("X2 shape (RGB):", X2.shape)

# -----
# Build Siamese model
# -----
model = build_siamese_network(input_shape=(224, 224, 3))
model.summary()

# -----
# Training
# -----

```



```

history = model.fit(
    [X1, X2],
    y,
    batch_size=16,
    epochs=20,
    validation_split=0.2,
    shuffle=True
)

# -----
# Save model and encoder weights
# -----
os.makedirs("model", exist_ok=True)

# Save full Siamese model
model.save("model/cheiloscopy_siamese_model.h5")

# Save encoder separately
encoder = model.get_layer("Encoder")
encoder.save_weights("model/encoder_weights.h5")

print("✅ Training completed and encoder weights saved")

```

4.Evaluation

```

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import load_model

# Load trained siamese model
model = load_model("model/cheiloscopy_siamese_model.h5", compile=False)

# Extract encoder
encoder = model.get_layer("Encoder")

# Load data
X1 = np.load(r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs\X1.npy")
X2 = np.load(r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs\X2.npy")
y = np.load(r"C:\Users\ADMIN\Desktop\cheiloscopy\pairs\y.npy")

# Add channel dimension
X1 = X1[..., np.newaxis]
X2 = X2[..., np.newaxis]

print("Computing embeddings...")

# ✅ PREDICT ONCE
embeddings_X1 = encoder.predict(X1, batch_size=32)
embeddings_X2 = encoder.predict(X2, batch_size=32)

print("Ranking evaluation started...")

```

```
def euclidean(a, b):  
    return np.linalg.norm(a - b)  
  
correct = 0  
  
for i in range(len(embeddings_X1)):  
    distances = [  
        euclidean(embeddings_X1[i], embeddings_X2[j])  
        for j in range(len(embeddings_X2))  
    ]  
    predicted_index = np.argmin(distances)  
    if y[predicted_index] == 1:  
        correct += 1  
  
rank_accuracy = correct / len(embeddings_X1)  
print(f"Rank-1 Accuracy: {rank_accuracy:.4f}")
```